

GATED

**Apartment Tracker: A Smart Residential Society
Management System**

Project Proposal for UCS503P

Submitted to:

Thapar Institute of Engineering and Technology

Submitted by:

Geetika Parakh
Ridham Sharma
Khushi Dangi
Harnoor Singh

August 19, 2026

1. Higher-Order Goal

The broader goal of this project is to improve the efficiency, transparency, and organisation of residential society management by reducing dependence on fragmented manual processes such as WhatsApp communication, spreadsheets, paper registers, and manual records.

The objective is to develop a centralised web-based platform that integrates common residential society workflows for residents, administrators, security guards, and maintenance staff with the aim of providing structured, role-specific access to various operations within a society.

2. Problem Statement

Residential societies frequently rely on disconnected, manual systems to manage day-to-day operations. This creates several issues:

- **Fragmented communication:** Requests and information are distributed across multiple channels.
- **Delayed complaint resolution:** Maintenance complaints may be difficult to assign and track.
- **Poor transparency:** Residents may not have a centralised view of complaint or payment status.
- **Manual visitor management:** Physical registers make visitor verification and record maintenance inefficient and cause delays.
- **Billing errors:** Manual generation and tracking of maintenance bills can lead to inconsistencies.
- **Booking conflicts:** Manual facility reservations can result in overlapping bookings.
- **Poor record management:** Society information is distributed across multiple systems, which makes it difficult to access.

These problems create a need for a centralised platform that can coordinate society operations while providing appropriate access to each stakeholder.

3. Objectives

The primary objectives of Apartment Tracker are:

- To digitise common residential society management activities.
- To provide a centralised platform for residents, administrators, security guards, and maintenance staff.
- To simplify complaint registration, assignment, and tracking.
- To improve visitor verification through digital visitor passes and QR codes.
- To provide transparency in maintenance billing and facility booking.
- To maintain centralised digital records of society operations.

- To improve operational efficiency through role-based workflows.
- To develop a modular architecture that can support future expansion.

4. Proposed Solution

4.1 Overview

Apartment Tracker will be developed as a centralised web-based residential society management system. It will provide different interfaces and permissions based on the user's role.

Residents will be able to perform everyday tasks like raise and track complaints, invite visitors, book facilities, view notices, and access maintenance bills. Administrators will manage residents, apartments, complaints, billing, facilities, parking, notices and operational information. Security guards will verify visitors and maintain entry and exit records, while maintenance staff will handle assigned complaints and update their status.

The system will integrate these activities through a common backend and centralised database, which will help reduce reliance on disconnected communication and manual record-keeping methods.

4.2 Core Modules

The proposed system will consist of the following major modules:

- **Authentication and Role Management** — secure login and role-based access.
- **Apartment and Resident Management** — management of flats, residents, ownership, and occupancy.
- **Complaint Management** — complaint submission, assignment, status tracking, and resolution.
- **Visitor Management** — visitor invitations, QR-based passes, and entry/exit records.
- **Maintenance Billing** — bill generation, payment status, and payment history.
- **Facility Booking** — availability checking and conflict-free booking.
- **Parking Management** — resident and visitor parking allocation.
- **Notice Board** — publication of society announcements and notices.
- **Notifications** — delivery of relevant updates to users.
- **Analytics Dashboard** — summaries of complaints, visitors, payments, and other operations.

4.3 Core Workflow

The system will follow a role-based workflow:

User Login → Role-Based Dashboard → Select Service → Submit Request/Action → Backend Validation → Database Update → Administrator/Staff Action → Status Update → User Notification

For example, the complaint workflow will follow:

Resident → Complaint Submitted → Administrator → Maintenance Staff Assigned → In Progress → Resolved → Closed

Similarly, visitor management will follow:

Resident → Visitor Invitation → QR Pass Generated → Security Verification → Entry Recorded → Exit Recorded

Facility booking will check the availability before confirming a reservation to prevent double booking.

5. Solution Approach

5.1 Web Architecture

Apartment Tracker will use a three-tier web architecture consisting of:

Presentation Layer → Business Logic Layer → Data Layer

The frontend will provide role-specific interfaces for different users. The backend will handle authentication, authorisation, business workflows, API requests, and validation. The database will store centralised records for users, apartments, complaints, visitors, bills, bookings, notices, and other system entities.

The architecture separates the major responsibilities of the system, making the application easier to develop, test, maintain and extend in future.

5.2 Technology Stack

The proposed technology stack is:

- **Frontend:** React.js, TypeScript, Tailwind CSS, React Router, Axios
- **Backend:** Node.js, Express.js
- **Database:** MongoDB Atlas
- **Authentication:** JWT, bcrypt
- **File Storage:** Cloudinary
- **Notifications:** Nodemailer

- **Deployment:** Vercel and Render

The selected technologies are based on the skillset of the team and are well-established web development tools, suitable for implementing the project's modular requirements.

5.3 Security and System Correctness

Security and correctness will be considered throughout the development process. The system will use JWT-based authentication, password hashing, role-based authorisation, input validation and file validation to maintain correctness. System-level constraints will also be enforced. Features like facility booking will validate slot availability to prevent double booking, while visitor QR codes will use controlled verification and expiry mechanisms to prevent any bypassers.

6. Feasibility Analysis

6.1 Technical Feasibility

The project is technically feasible because its core functionality can be implemented with well-developed web technologies. React, Node.js, Express, MongoDB, JWT and supporting libraries provide the capabilities needed for primary processes like authentication, database management, REST APIs, QR code generation, file handling and dashboard development.

The project does not require specialised hardware or experimental algorithms for its core implementation. Its modular structure also allows different components to be developed and tested incrementally.

Cloud deployment with Vercel, Render, and MongoDB Atlas further reduces the academic prototype's infrastructure requirements.

Conclusion: The project is technically feasible with the proposed technology stack and infrastructure.

6.2 Economic Feasibility

The project is economically feasible because development will primarily rely on open-source technologies and widely available development tools. The proposed architecture does not require dedicated physical servers or expensive proprietary software for the academic prototype.

Cloud services with suitable free or development tiers can be used during development and testing, keeping initial infrastructure requirements low.

Conclusion: The project can be developed with relatively low infrastructure and software costs and is economically feasible as an academic project.

6.3 Operational Feasibility

The proposed system is operationally feasible because it is working on digitising activities that already exist in residential societies rather than introducing entirely new processes.

Residents can continue to perform familiar day-to-day activities, such as filing complaints, inviting visitors, booking facilities and viewing bills, through a centralised interface. Administrators, security guards and maintenance staff receive interfaces corresponding to their existing responsibilities.

The system, therefore, aims to reduce manual effort while improving coordination and transparency, without fundamentally changing stakeholders' roles.

Conclusion: The system is operationally feasible and can support all the major existing workflows of a residential society.

6.4 Schedule Feasibility

The project is planned as a 6-month semester-long development effort. Development will be carried out incrementally with regular updates on all core concepts weekly.

To control project scope, core functionality will be prioritised over advanced features. This allows a working MVP to be completed even if some secondary modules require additional development time.

Conclusion: The project is schedule-feasible provided that the MVP is prioritised and development progresses incrementally.

6.5 Security and Legal Feasibility

The system will handle information such as resident details, visitor information, complaints, images, and billing records. Therefore, only information necessary for system functionality should be collected and stored.

Authentication, authorisation, password hashing, input validation and file validation will be used to protect system resources. Visitor information will be handled with controlled access, and QR-based visitor passes will include verification and expiry mechanisms. This will avoid any breach of private data.

The system is intended initially as an academic prototype only.

Conclusion: The project is security-feasible, provided appropriate data-handling, authentication, authorisation, and validation practices are followed.

7. Evaluation Criteria

7.1 Primary Evaluation Metrics

The primary evaluation of the system will focus on whether it improves the efficiency of important society workflows.

The following metrics will be considered:

- **Complaint processing time:** Time between complaint submission and assignment/status update.
- **Visitor verification time:** Time required to verify a visitor using the QR-based process.
- **Booking conflict rate:** Number of conflicting facility bookings after system implementation.

7.2 Secondary Evaluation Metrics

Additional metrics will include:

- API response time for common operations.
- System availability during the evaluation period.
- Successful completion rate of major user workflows.
- User-reported ease of use and clarity.
- Number of manual steps eliminated from selected workflows.

7.3 Validation Plan

The system will be evaluated using representative users or simulated stakeholder roles, including residents, administrators, security guards, and maintenance staff.

Users will perform common workflows such as submitting a complaint, verifying a visitor, booking a facility, and viewing a bill. System logs will be used to measure workflow performance, while user feedback will be collected to evaluate usability and clarity.

The evaluation will compare the effectiveness of the centralised workflows against the corresponding manual processes.

8. Scalability

The system will be designed with scalability in mind, so that increases in residents, complaints, visitors, bookings, and other records do not require major architectural changes.

The separation of frontend and backend components will allow individual layers to be scaled independently. Database indexing and efficient queries can support frequent access to information, while pagination can be used for large lists such as complaints, visitors, and activity records.

The stateless nature of REST APIs and cloud-based deployment will also support the system's future expansion.

The modular architecture allows additional functionality, such as mobile applications, AI-based classification, IoT integration, and automated security systems to be added in future iterations.

9. Project Scope and Deliverables

9.1 Initial Deliverable

The initial working version will prioritise the core society workflows:

- User authentication and role-based access.
- Apartment and resident management.
- Complaint submission and tracking.
- Complaint assignment and status updates.
- Visitor invitation and QR-based verification.
- Facility availability and booking.
- Basic notice management.
- Centralised database and REST APIs.
- Basic logging and system validation.
- Deployment of the working application.

9.2 Subsequent Deliverables

Subsequent iterations will extend the system with:

- Maintenance billing and payment tracking.
- Parking management.
- Notifications.
- Analytics dashboard.
- Improved search and filtering.
- Additional reporting and administrative features.

The division between initial and subsequent deliverables allows the team to prioritise a functional core system while retaining scope for additional features.

10. Risks and Mitigations

- **Scope expansion:** The number of proposed modules may make it difficult to complete the project within the semester. This will be mitigated by defining an MVP and incrementally implementing secondary modules.
- **Role-based access errors:** Incorrect permissions could expose restricted functionality. Role permissions will therefore be defined during system design and tested independently.
- **Facility booking conflicts:** Concurrent booking requests could create double bookings. Availability will be checked and validated at the backend before confirmation.
- **QR security:** Visitor passes could be misused if they remain valid indefinitely. QR passes will use controlled verification and expiry mechanisms.
- **Data security:** Resident and visitor information must be protected. Authentication, password hashing, authorisation, input validation, and controlled data collection will be implemented.
- **Integration issues:** Multiple modules depend on the same database and backend services. Modular API design and integration testing will be used to identify issues early.
- **Deployment problems:** Deployment will be attempted before the final development stage to prevent infrastructure issues from delaying the final demonstration.

11. Summary

Our Apartment Tracker is a centralised web-based solution to manage the functioning and day-to-day activities of a residential society, which are currently handled through manual methods, to improve efficiency, prevent delays and add ease to the overall experience of the users. It will act as a link between residents, administrators, security guards and maintenance staff, helping in managing everyday functioning.

It is a feasible solution, including technical and operational feasibility, with a primary focus on delivering a functional, deployable system following an incremental development from an academic point of view, to help us learn all core concepts of software development.

The evaluation matrices for the project will be based on the workflow of different operations, performance of different features and overall ease of usability.

Overall, this makes the project manageable while demonstrating key concepts.